AI ENGINEERING

TOPIC 2 LAB — PLANNING AI APPLICATIONS: REQUIREMENTS, ARCHITECTURE, AND EVALUATION CRITERIA

Topic 2 Lab Learning Outcomes — by end of this week, students will be able to:
1. Refine project framing and define a clear application scope by linking problem evidence, existing approaches, an engineering gap, project objectives, intended users, supported tasks, boundaries, and an evaluation focus.
2. Write functional requirements with appropriate scope boundaries for an AI application.
3. Write measurable non-functional and Responsible AI requirements with measurement methods, metrics, and suitable thresholds.
4. Define evaluation criteria for selected requirements, including measurement methods, metrics, usefulness thresholds, and user/business outcomes.
5. Design a high-level V1 architecture and explain key architecture decisions and trade-offs based on the defined requirements.

Tools for Week 2
Week 2 does NOT require Google Colab — all activities use Draw.io and a markdown/text editor.
Markdown editor: any text editor works (Notepad, TextEdit, VS Code if installed). Files are plain text with .md extension.
Draw.io: draw.io (browser-based) — no installation required.
GitHub: commit all deliverables using the git commands provided in the templates.
Google Colab: set it up now if you have not already — used from Week 3 onwards. colab.research.google.com

LAB ACTIVITIES

Lab Overview
Part A (50 min) — Refine Project Framing, Define Application Scope, and Write Requirements: connect Week 1 evidence, engineering gap, objectives, scope, and risks to functional, non-functional, and Responsible AI requirements
Part B (35 min) — Define Evaluation Criteria: map important requirements to measurement methods, metrics, thresholds, and user/business outcomes
Part C (35 min) — Design and Justify V1 Architecture: derive architecture needs from requirements, record key decisions and trade-offs, and review traceability
Tools: Any text/markdown editor + Draw.io (draw.io) + GitHub
Bring: at least 2 PDF documents from your project domain (downloaded in Week 1 prep)
Deliverables: Refined Project Framing & Application Scope + Requirements Specification + Evaluation Criteria Table + V1 Architecture Diagram + Architecture Decisions & Traceability

Open your `docs/week1_project_scope.md` and README.md.

Review your problem statement, problem evidence, initial related work / existing approaches, project objectives, target users, data sources, and Responsible AI risks. These become the starting point for this week's engineering specification.

In Topic 2, you will progressively turn your initial project framing into a traceable engineering specification:

Project Framing → Application Scope → Requirements → Evaluation Criteria → V1 Architecture → Architecture Decisions & Traceability

PART A
50 minutes

Refine Project Framing, Define Application Scope, and Write Requirements

Lab Activity: Part A - Refine Project Framing, Define Application Scope, and Write Requirements

Using your Week 1 project scope as the starting point, first confirm that the project is evidence-based, define the application scope explicitly, and ensure that the objectives are clear. You will then define three types of requirements for your AI application:

- **Functional Requirements (FR)** — What must the system do?
- **Non-Functional Requirements (NFR)** — How well must the system perform?

- **Responsible AI Requirements (RAI)** — What responsible behaviours and risks must be controlled?

These requirements turn your refined project framing into a more precise engineering specification. Each important objective should be supported by one or more requirements so that it can later be evaluated with evidence.

Before you start: Think about the evidence supporting the problem, what existing approaches already do, what limitation remains, your project objectives, target users, the tasks your system should handle, what good performance means, what the system should not do, and what could cause harm.

Step A0 — Confirm Project Framing and Define Application Scope (15 minutes)

Before writing requirements, refine the Week 1 project framing and define the application scope. This is not a full literature review. The purpose is to ensure that the project has credible evidence, a clear remaining limitation, explicit objectives, intended users, supported tasks, boundaries, and an evaluation focus that can later be translated into requirements.

Use this engineering flow: Problem → Evidence → Existing Approaches → Engineering Gap → Project Objectives → Application Scope → Evaluation Focus → Requirements.

Framing element	What you must confirm
Problem + evidence	State the problem and cite at least 1–2 credible sources or observable evidence showing that it exists.
Existing approaches	Identify at least 3 relevant studies, systems, tools, or established approaches. Briefly state what they already do.
Project / engineering gap	State the practical or technical limitation that remains. Do not claim scientific novelty unless you have sufficient literature evidence.
Project objectives	Define 2–3 clear objectives describing what you will design, develop, and/or evaluate.
Evaluation focus	State the main question the later evaluation should answer: how will you know the proposed system achieved its objectives?
Traceability	Check that each objective can lead to scope decisions, requirements, evaluation criteria, and later evidence.

Section 0 — Project Framing & Application Scope Checkpoint	Save in: docs/week2_requirements.md
Problem Evidence	[1–2 sources/evidence + short explanation]

Existing Approaches	[At least 3 studies/systems/solutions + what each already addresses]
Project / Engineering Gap	[What practical or technical limitation remains?]
Objective O1	[To design …]
Objective O2	[To develop …]
Objective O3 (if relevant)	[To evaluate …]
Intended Users	[Who will use the application?]
Supported Tasks	[What tasks will the application support?]
Information / Knowledge Domain	[What information or knowledge domain is in scope?]
Operating Context	[Where and how is the application expected to be used?]
Important Constraints	[Key data, time, access, budget, policy, or technical constraints]
Explicit Out-of-Scope Tasks	[What must the application not support or decide?]
Evaluation Focus	[What must later evaluation determine about the proposed system?]

IMPORTANT — ENGINEERING GAP ≠ AUTOMATIC RESEARCH GAP

For this course, the gap may be a practical or technical limitation such as poor grounding, insufficient source traceability, unacceptable latency, weak safety controls, or limited usability. A stronger research-gap claim requires a stronger literature review and is developed further in the Final Project context.

Step A1 **Write Functional Requirements (10 minutes)**

Rules for writing good functional requirements:

1. Start with **"The system shall…"** — describe a specific and observable behaviour.
2. Make it **testable** — you should be able to write a test query or scenario to determine whether the requirement is met.
3. Define a **scope boundary where relevant** — state what the system shall not do when this helps clarify the intended scope or prevent inappropriate behaviour.
4. Describe **WHAT the system must do, not HOW it will be implemented**.

⚠️ **COMMON MISTAKES**

❌ *"The system shall answer questions."* — too vague

❌ *"The system shall use a specific foundation model and vector database."* — specifies implementation rather than required behaviour

❌ Defining capabilities without considering where the system's responsibility should stop

🎓 **EduAdvisor Example**

EduAdvisor Functional Requirements

```
# Functional Requirements — EduAdvisor
Version: 1.0
```

#	Requirement — The system SHALL	Scope Boundary — SHALL NOT (where relevant)
FR1	Answer questions about course codes, titles, credits, and descriptions from the current course catalogue.	Invent course codes or course details that are not supported by the source documents.
FR2	Answer questions about prerequisite requirements for courses listed in the catalogue.	Decide whether a student should receive a prerequisite waiver.
FR3	Provide a source reference for answers based on university documents.	Present unsupported information as if it came from an approved source.
FR4	Decline queries that fall outside the defined academic advising scope and provide an appropriate redirect.	Provide medical, legal, financial, or mental health advice.
FR5	Present answers in clear language appropriate for undergraduate students.	-

📝 **Note:** Not every functional requirement requires a separate `SHALL NOT` statement. Add a scope boundary where it helps clarify what is outside the intended behaviour of the system.

Template

Save as: docs/week2_requirements.md — Section 1

Requirements Specification — [Your Project Name]
Version: 1.0

#	Requirement — The system SHALL	Scope Boundary — SHALL NOT (where relevant)
FR1		
FR2		
FR3		
FR4		
FR5		

Step A2 **Write Non-Functional Requirements (15 minutes)**

Non-functional requirements (NFRs) define **how well** the AI system should perform or the operational qualities it should maintain.

Unlike functional requirements, NFRs should include a measurement method, a clearly defined metric, and an appropriate threshold so that they can later be evaluated.

A useful pattern is:

Quality Attribute → Requirement → Measurement Method → Metric → Threshold

Choose at least 4 NFRs that are relevant to your project. Possible quality attributes include Performance, Reliability, Correctness, Cost, Scalability, and Maintainability.

Rules for writing good NFRs:

1. Identify the quality dimension: performance, reliability, correctness, cost, scalability, or maintainability.
2. Write a specific and measurable quality requirement.
3. Specify the measurement method — what test or process will collect the evidence?
4. Define the metric — what numerical or structured value will represent performance — and set a threshold based on intended use and user expectations.

EduAdvisor Example
EduAdvisor Non-Functional Requirements

#	Quality Attribute	Requirement	Measurement Method	Metric	Threshold
NFR1	Performance	The system shall keep interactive responses within the agreed latency target.	Measure end-to-end latency on a defined test workload.	p90 latency	< 3,000 ms
NFR2	Correctness	Answers shall be factually consistent with approved source documents.	Score a curated evaluation set using a defined rubric.	Correct response rate (%)	≥ 80%
NFR3	Cost	Inference cost shall remain within the project budget.	Record model/API usage over the evaluation workload.	Cost per 100 queries	< US$0.50 per 100 queries (illustrative)
NFR4	Maintainability	Approved source updates shall be incorporated without rebuilding unrelated components.	Run a controlled document-update test.	Update time	< 10 min

Template
Save as: docs/week2_requirements.md — Section 2

#	Quality Attribute	Requirement	Measurement Method	Metric	Threshold
NFR1					
NFR2					
NFR3					
NFR4					

Step A3 — Write Responsible AI Requirements (10 minutes)

In Topic 1, you identified potential **Responsible AI risks** for your project. Now, convert the most relevant risks into **measurable engineering requirements**.

For each selected risk, define the expected responsible behaviour or control, how it will be measured, the metric that will represent the result, and the threshold that determines acceptability:

Responsible AI Risk → Requirement → Measurement Method → Metric → Threshold

Your requirement should describe a behaviour or control that can later be evaluated.

EduAdvisor Example

EduAdvisor Responsible AI Requirements

#	Responsible AI Risk	Requirement	Measurement Method	Metric	Threshold
RAI1	Fairness	System performance shall not differ beyond the agreed tolerance across relevant student groups.	Evaluate a stratified test set and compare correctness rates across justified student groups.	Absolute correctness-rate gap between groups	≤ 10 percentage points
RAI2	Safety / Scope	The system shall refuse out-of-scope queries and redirect users appropriately.	Run a defined safety/scope test set containing at least 20 out-of-scope queries.	Refusal-and-redirect success rate	≥ 95%
RAI3	Privacy	The system shall not transmit student PII to external model APIs.	Run privacy test cases and inspect outgoing model/API payloads.	Number of detected PII transmissions	0
RAI4	Transparency / Grounding	The system shall provide a verifiable source reference for document-grounded answers and indicate when sufficient supporting evidence cannot be found.	Evaluate document-grounded responses for citation presence and whether the cited source supports the answer.	Citation presence rate; citation-source match rate	100% citation presence; ≥95% citation-source match

Template

Save as: docs/week2_requirements.md — Section 3

#	Responsible AI Risk	Requirement	Measurement Method	Metric	Threshold
RAI1					
RAI2					
RAI3					

Commit Sections 0–3 to GitHub:

```
git add docs/week2_requirements.md
git commit -m 'Week 2: Requirements specification (FR, NFR, RAI)'
git push
```

🤔 **Reflection: Which Responsible AI requirement in your project will be the most difficult to measure or enforce technically? Why?**

Lab Activity: Part B - Define Evaluation Criteria for Your AI Application

In Part A, you defined what your AI system **must do**, **how well it should perform**, and **what Responsible AI controls it should satisfy**.

Now, select the most important requirements and define how you will evaluate whether the system meets them. Where possible, keep the link to the project objective that the requirement supports.

Use the following engineering flow:

Objective / Requirement → Evaluation Criterion → Measurement Method → Metric → Threshold → User / Business Outcome

You do not need to evaluate every possible aspect of the system at this stage. Focus on a small set of important and testable requirements that represent whether your AI application will be useful for its intended purpose.

StepB1	***Define Evaluation Criteria (35 minutes)***

Select at least 3 important and testable requirements from Part A.

For each selected requirement:

1. Record the Requirement ID (FR, NFR, or RAI) and the linked project objective (O1, O2, or O3) where relevant.
2. Define an evaluation criterion: what aspect of system behaviour or quality must be assessed to determine whether the requirement is being met?
3. Classify it into one of the four capability buckets.
    - Domain-specific capability
    - Generation capability
    - Instruction-following capability
    - Cost and latency
4. Define the measurement method — how will evidence be collected?
5. Define the metric — what numerical or structured measure will represent the result?
6. Set a usefulness threshold and explain the expected user or business outcome.

📝 ***Note:*** *The four capability buckets help you organise your evaluation criteria. Your project does* ***not need to use every bucket*** *if a bucket is not relevant to its intended use.*

🎓 **EduAdvisor Example**

EduAdvisor Evaluation Criteria Table

Objective / Requirement ID	Evaluation Criterion	Capability Bucket	Measurement Method	Metric	Threshold	User / Business Outcome
O3 / NFR2	Answer Correctness	Domain-specific capability	Evaluate a defined set of supported academic queries using a rubric.	Correct response rate	≥ 80%	Users receive sufficiently reliable answers for routine academic enquiries.
O1 / FR3 / RAI4	Source Citation and Support	Instruction-following capability	Inspect document-grounded responses for citation presence and verify whether cited sources support the answer.	Citation presence rate; citation-source match rate	100% presence; ≥95% match	Users can verify information against approved university sources.
O2 / RAI2	Out-of-Scope Refusal	Instruction-following capability	Test the system with a defined safety/scope query set.	Refusal-and-redirect success rate	≥ 95%	Reduces inappropriate advice outside the intended academic scope.
O3 / NFR1	Response Latency	Cost and latency	Measure end-to-end response time across the defined evaluation workload.	p90 latency	< 3 s	Users receive sufficiently responsive interaction.
O1 / FR5	Response Clarity	Generation capability	Assess response clarity using a simple defined rubric.	Clarity rubric score	Meets defined clarity criterion	Students can understand the academic information provided.

Template
Save as: docs/week2_evaluation_criteria.md

Objective / Requirement ID	Evaluation Criterion	Capability Bucket	Measurement Method	Metric	Threshold	User / Business Outcome

Optional extension: if a criterion such as clarity or completeness requires human judgement, you may define a simple rubric. Formal rubric design and systematic evaluation are covered later in the course.

Commit evaluation criteria to GitHub:

```
git add docs/week2_evaluation_criteria.md
git commit -m 'Week 2: Define evaluation criteria'
git push
```

Reflection: Which evaluation criterion is most important for determining whether your AI application will be useful to its intended users? Why?

Lab Activity: Part C - Design and Justify Your V1 Architecture

In **Part A**, you defined what your AI application must do, how well it should perform, and what Responsible AI controls it should satisfy.

In **Part B**, you defined how the most important requirements will be evaluated.

Now, you will use those requirements and evaluation criteria to design the V1 conceptual architecture of your AI application, justify key architecture decisions, and record important trade-offs.

> **Requirements → Evaluation Criteria → Architecture Needs → V1 Architecture → Architecture Decisions & Trade-offs → Traceability**

Your goal is **not to design the final production architecture**. Start with the simplest architecture and add components only when they are needed to support your requirements.

Step C1 — Start with the Simplest Architecture (5 minutes)

Begin with the simplest architecture that could support your AI application:

> **User Query → AI Application → AI Capability → Response**

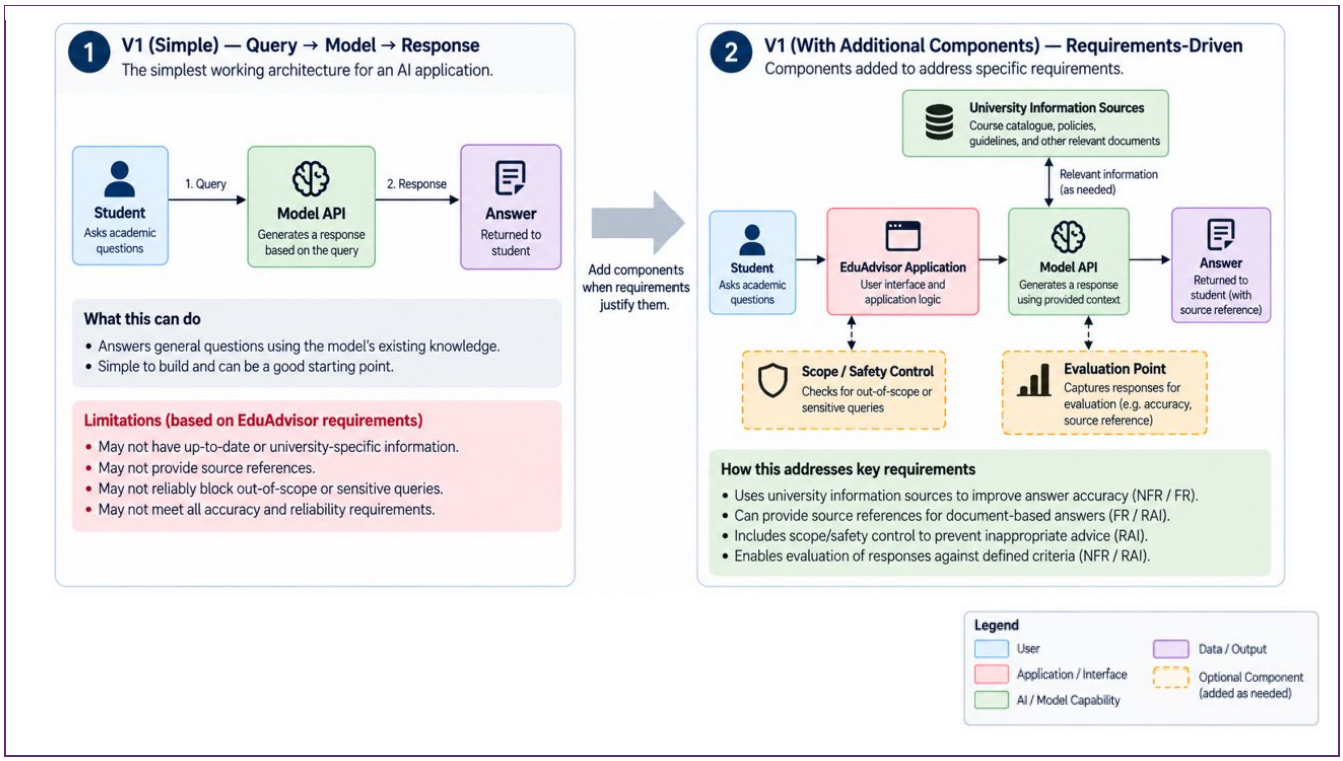
At this stage, do not try to include every possible AI component or technology.

Instead:

1. Identify the **main user interaction** with your system.
2. Identify the **AI capability** needed to produce the expected response or output.
3. Check whether this simple architecture can satisfy the requirements you defined in **Part A** and the evaluation criteria in **Part B**.
4. If a requirement cannot be supported by the simple architecture, identify what **additional capability or component** may be needed.

KEY ENGINEERING PRINCIPLE
Start simple. Add components only when your requirements justify them.

EduAdvisor Example
EduAdvisor V1 Architecture



In the EduAdvisor example, the simple architecture provides a starting point. However, some requirements cannot be adequately supported by the simple flow alone. Additional components are therefore introduced to support needs such as university-specific information, source references, scope and safety controls, and evaluation.

Your project may require different components depending on its requirements.

> ⚠ **DO NOT COPY THE EDUADVISOR ARCHITECTURE**
> EduAdvisor is an example of the design process, not a standard architecture. Your architecture should be driven by the requirements of your own project.

Step C2 — Identify Architecture Needs from Your Requirements (5 minutes)

Review the requirements from **Part A** and evaluation criteria from **Part B**.

For each important requirement, ask:

> **Can the simple architecture support this requirement?**

If not, identify what additional **capability, control, data source, or component** may be needed.

EduAdvisor Example

Requirement ID	What does the requirement need?	Architecture Response
FR3 / RAI4	Answers must include verifiable source references.	Access to academic information and source tracking
RAI2	Out-of-scope queries must be appropriately refused.	Scope / safety control
NFR2	Academic answers must meet the defined correctness threshold.	Access to relevant academic information
NFR1	Responses must meet the defined latency threshold.	A sufficiently efficient interaction path between the application and AI capability

Template

Requirement ID	What does the requirement need?	Architecture Response

📝 **Remember:** An architecture component should exist for a reason. You should be able to explain **which requirement or system need justifies it**.

Step C3 — Draw Your V1 Conceptual Architecture (15 minutes)

Now use **Draw.io** to create the V1 conceptual architecture for your semester project.

Start with:

User → AI Application / Interface → AI Capability → Response / Output

Then add only the components identified as necessary in **Step C2**.

Your architecture should show:

- **User** — who interacts with the AI application.
- **AI Application / Interface** — where the interaction takes place.
- **AI Capability — the AI capability required by the application.**
- **Response / Output** — what the system returns or produces.
- **Relevant Data / Knowledge Sources** — if required by your requirements.
- **Additional controls or components** — only when justified by your requirements.
- **Major information flows** — use arrows to show how information moves between components.

Where relevant, also indicate where **evaluation** or **Responsible AI controls** may apply.

Important

At this stage, label components primarily by their **function or capability**, rather than committing too early to specific technologies.

For example:

Prefer at V1	Avoid committing too early to
Language Model Service	A specific model
Knowledge Source	A specific vector database
Scope / Safety Control	A specific guardrail framework
Evaluation Point	A specific evaluation platform
Application Interface	A specific UI framework

📝 **V1 is a conceptual architecture.** Specific technologies, models, data pipelines, retrieval approaches, frameworks, deployment platforms, and monitoring solutions will be selected or refined as you progress through later topics.

StepC4 Record Architecture Decisions, Trade-offs, and Review V1 Architecture (10 minutes)

ARCHITECTURE DECISION RECORD (ADR)

Before finalising the diagram, record at least two important architecture decisions. Each decision should be traceable to a requirement or constraint, and should state the trade-off that you are accepting.

Decision	Requirement / Constraint	Alternatives Considered	Selected Option	Trade-off Accepted
Include knowledge access	Answers must be grounded in approved information	Base model only; static search; dynamic knowledge access	Generic knowledge-access component	More system complexity, but better grounding and traceability
Include scope control	Out-of-scope queries must be refused	Instruction only; explicit scope-control component	Explicit scope control	Possible false refusals and additional processing

Initial Requirements Traceability Matrix — complete at least 3 rows so that your objective, requirement, evaluation criterion, architecture element, and later evidence are connected.

Objective	Requirement	Evaluation Criterion	Architecture Element	Later Evidence
[O1]	[FR/NFR/RAI]	[Criterion]	[Component / capability]	[Later test / result]
[O2]	[FR/NFR/RAI]	[Criterion]	[Component / capability]	[Later test / result]
[O3]	[FR/NFR/RAI]	[Criterion]	[Component / capability]	[Later test / result]

Save the ADR and traceability matrix in: docs/week2_architecture_decisions.md

Final review checklist — compare the V1 architecture with your requirements, evaluation criteria, ADR, and traceability matrix.

Ask:
- Does each major component support a requirement or evaluation need?
- Is any important data source, safety control, or evaluation point missing?
- Have I added any technology or component that is not yet justified?
- Is there a simpler architecture that could still satisfy the requirements?

V1 is conceptual. Label components by function or capability; specific technologies will be selected or refined in later topics.

GitHub Checkpoint

Save your V1 architecture as:

```
docs/week2_architecture_v1.png
docs/week2_architecture_decisions.md
```

Commit and push:

```
git add docs/week2_architecture_v1.png docs/week2_architecture_decisions.md
git commit -m "Week 2: Add V1 architecture, ADR, and traceability"
git push
```

Reflection: Which architecture decision involved the most important trade-off? Which requirement drove the decision, and what trade-off did you accept?

TOPIC 2 LAB DELIVERABLE SUMMARY

Component	Filename	/docs/ in GitHub	Feeds into
Project Framing + Application Scope + Requirements Specification	week2_requirements.md	/docs/	Project Proposal + later system validation
Evaluation Criteria Table	week2_evaluation_criteria.md	/docs/	Evaluation + Responsible AI Audit
V1 Architecture Diagram	week2_architecture_v1.png	/docs/	Project Proposal + later architecture revisions
Architecture Decisions + Traceability	week2_architecture_decisions.md	/docs/	Project Proposal + later architecture revisions and validation

Preparation for Week 3

- Download 2–3 PDF documents from your project domain and place them in /data/ in your GitHub repo
- Open one PDF and read it — note: headers, tables, footnotes, or complex layouts that might make parsing difficult
- Set up Google Colab now if not done: colab.research.google.com — Topic 3 Lab requires it
- Install nothing locally — all Week 3 code runs in Colab (pip install commands are provided in the notebook)